Set_A_Lab4 - Results

Attempt 1 of 1

Written Feb 4, 2026 9:42 PM - Feb 4, 2026 11:30 PM

Attempt Feedback

30% penalty
15/17

Submitted Late: 4 hours past the due date

Attempt Score	**10.5 / 17 - 61.76 %**
Overall Grade (Highest Attempt)	**10.5 / 17 - 61.76 %**

Question 1

First compile your code (https://cocalc.com/) to obtain main1.out and then run main1.out using:

./main1.out & pstree -pT | grep main1.out

main1.c

```c
#include <stdio.h>
#include <unistd.h>

int main(){

    pid_t fork_return = fork();
    pid_t  pid = getpid();
    printf("fork return value = %d , pid = %d \n", fork_return, pid);
    sleep(1);

    return 0;
}
```

Answer the following questions (Q2, Q3, Q4, Q5):

Insert a screenshot of your program's output after execution.

```
~$  ./main1.out & pstree -pT | grep main1.out
[2] 488
fork return value = 491 , pid = 488
fork return value = 0 , pid = 491
                          |           |-main1.out(488)---main1.out(491)
[1]   Done                    ./main1.out
```

chrome_KmwibfztAT.png (4.73 KB)

The correct answer is not displayed for Written Response type questions.

Question 2

What is the main purpose of the **`getpid()`** function?

- ✔ (●) To get the process ID of the current process
- ( ) To get the process priority
- ( ) To get the thread ID
- ( ) To get the parent process ID

Question 3

What type of value does getpid() return?

- ○ A memory address
- ○ A floating-point number
- ✔ ◉ An integer representing the process ID
- ○ A string containing the process name

Question 4

Refer to your results from **Question 1** and select the correct option describing the parent process (**PID_P**) and child process (**PID_C**).

- ○ $PID_P = PID_C$
- ○ $PID_P = 0$ and $PID_C > 0$
- ✔ ◉ $PID_P \neq PID_C$
- ○ $PID_P > 0$ and $PID_C = 0$

Question 5

Based on the results of Question 1, choose the correct fork() return value for both the parent (**PID_P**) and child (**PID_C**) processes

✔ __3__	In the parent process, the return value of fork() is	**1.** 0

✔ __1__ In the child process, the return value of fork() is

2. PID_P
3. PID_C
4. -1

Question 6

Consider the following C code, first compile your code to obtain the binary file main2.out (use https://cocalc.com/) and then run it.

gcc -static main2.c -o main2.out

main2.c

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

int global_variable = 100;
int un_global_variable;

int main()
{
    char str[30];
    int *ptr;
    int local_variable = 200;
    int un_local_variable;

    ptr = (int*) malloc(2 * sizeof(int));

    pid_t pid;
    pid = getpid();

    printf("\n");
    printf(" Global variable                 : %p\n", &global_variable);
    printf(" Uninitialized global variable   : %p\n", &un_global_variable);

    printf(" Local variable                  : %p\n", &local_variable);
    printf(" Uninitialized local variable    : %p\n", &un_local_variable);

    printf(" Memory allocation               : %p\n\n", ptr);
    printf("  ======== The Process Address Space ( pid = %d )=======\n\n", pid);

    sprintf(str, "cat -b /proc/%d/maps", pid);
    system(str);
    free(ptr);

    return 0;
}
```

Insert a screenshot of your output.

```
~$ ./main2.out

 Global variable                : 0x4ac0d0
 Uninitialized global variable  : 0x4adb30
 Local variable                 : 0x7ffd26e7b56c
 Uninitialized local variable   : 0x7ffd26e7b570
 Memory allocation              : 0x1197d900

 ======== The Process Address Space ( pid = 661 ) ========

    1  00400000-00401000 r--p 00000000 00:1cb 388                          /home/user/main2.out
    2  00401000-00481000 r-xp 00001000 00:1cb 388                          /home/user/main2.out
    3  00481000-004a7000 r--p 00081000 00:1cb 388                          /home/user/main2.out
    4  004a7000-004ac000 r--p 000a6000 00:1cb 388                          /home/user/main2.out
    5  004ac000-004ae000 rw-p 000ab000 00:1cb 388                          /home/user/main2.out
    6  004ae000-004b4000 rw-p 00000000 00:00 0
    7  1197c000-1199e000 rw-p 00000000 00:00 0                            [heap]
    8  7ffd26e5b000-7ffd26e7d000 rw-p 00000000 00:00 0                    [stack]
    9  7ffd26f75000-7ffd26f79000 r--p 00000000 00:00 0                    [vvar]
   10  7ffd26f79000-7ffd26f7b000 r-xp 00000000 00:00 0                    [vdso]
   11  ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0            [vsyscall]
```

chrome_pw0xsaFUsZ.png (21.95 KB)

The correct answer is not displayed for Written Response type questions.

Question 7

Refer to your results from **Question 6** and obtain the location of the initialized and uninitialized global variables (the line number)? e.g. 12 ___5___ ✔(33.33 %)

Refer to your results from **Question 6** and obtain the location of the initialized and uninitialized local variables (the line number)? e.g. 12 ___8___ ✔(33.33 %)

Refer to your results from **Question 6** and obtain the location of allocated memory (the line number)? e.g. 12 ___7___ ✔(33.33 %)

e.g.

```
 line 34

33  7ffc7e75b000-7ffc7e75f000 r--p 00000000 00:00 0                    [vvar]
34  7ffc7e75f000-7ffc7e761000 r-xp 00000000 00:00 0                    [vdso]
35  ffffffffff600000-ffffffffff601000 r-xp 00000000 00:00 0            [vsyscall]
```

Question 8

A computer system can hold three identical processes in its main memory at a time. Each process has an independent probability p = 80%.

Calculate the overall CPU utilization, defined as the probability that at least one process is ready to use the CPU. e.g. 0.123 ___0.973___ ✖ **(0.488)**

Now consider only the scenario in which exactly one process is waiting for I/O, and the other two are ready. What is the CPU utilization in this specific case? e.g. 0.123 ___1.000___ ✖ **(0.096)**

Question 9

Consider the following c code, compile your code to obtain the binary file main3.out.

gcc main3.c -o main3.out

main3.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Process x \n");
        exit(0);
    } else {
        printf("Process y \n");
        sleep(30);
    }

    return 0;
}
```

Run your program and use its output to answer the following questions:

./main3.out & ps -e -o pid,stat,comm | grep main3.out

Insert a screenshot of your program's output.

```
~$  ./main3.out & ps -e -o pid,stat,comm | grep main3.out
[1] 777
Process y
Process x
    777 SN   main3.out
    780 ZN   main3.out
```

chrome_3dxKOanfue.png (3.32 KB)

The correct answer is not displayed for Written Response type questions.

Question 10

Refer to your results from **Question 9,** the child process is e.g. x or y

___x___ ✔(50 %)

Refer to your results from **Question 9,** the parent process is e.g. x or y

___y___ ✔(50 %)

Question 11

Refer to your results from **Question 9**, Specify the process state of both the child and the parent.

✔ __4__ Process state of Child

✔ __2__ Process state of Parent

1. **T**
2. **S**
3. **D**
4. **Z**

5. R

Done